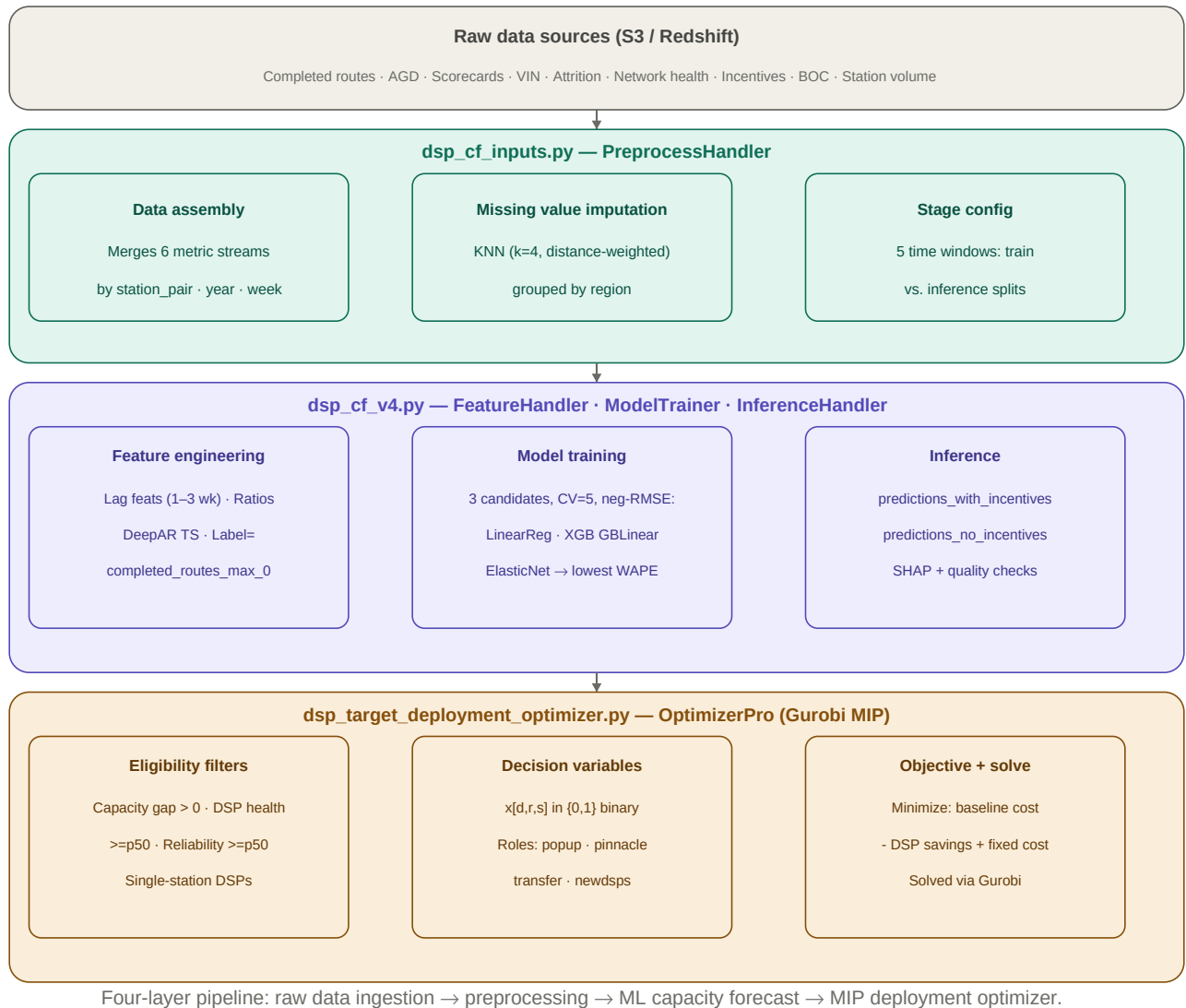


DSP Peak Scaling & Target Deployment

System architecture, ML model, and optimization objective — code walkthrough

System architecture overview



What this system does

This is an **Amazon Last Mile (AMZL) workforce planning system** that forecasts how many delivery routes a DSP (Delivery Service Partner) can handle at a given station, then optimizes which DSPs should be recruited or redeployed to close capacity gaps at under-served stations. It runs on AWS (SageMaker, Redshift, S3) with Gurobi as the MIP solver, and supports peak season planning (Q4 holiday surge).

File 1 — dsp_cf_inputs.py

Data ingestion and preprocessing pipeline

This is the ETL layer. **PreprocessHandler** pulls from ~10 different Redshift/S3 sources and produces a clean parquet file (*dsp_peak_scaling_input_data.pqt*) with ~80 features per DSP x station x week row.

Data sources merged

- Completed routes, AGD (assigned-to-delivered)
- Grounded vans, ROBL (route-off-block time), scorecards
- BOC (books of complaints), incentives, VIN (vehicle utilization)
- Attrition, network health scores

All keyed on **station_pair** (station + DSP code), year, and week.

Key processing steps

- **KNN imputation** (k=4, distance-weighted) grouped by region — fills missing feature values using similar DSPs in the same geographic cluster
- **Year-lagged imputation** as fallback for DSPs without multi-year continuity
- **Stage configuration generation** — splits history into 5 time windows (stages 1–5) defining training vs. recent performance vs. prior-peak contexts

Data quality gates

- Raises exception if DSP data < 600 rows, VIN/attrition data < 200 rows, or final input < 200 rows after merging

File 2 — dsp_cf_v4.py

Capacity forecast ML model

The core prediction engine. Contains three major handler classes: **FeatureHandler**, **ModelTrainer**, and **InferenceHandler**.

FeatureHandler — feature engineering

- Lag features at 1, 2, 3, and 4 weeks back for key metrics (e.g. `reliability_1`, `dsp_nhscore_3`)
- Station-level ratio features: a DSP's share of station routes at various lags
- GluonTS DeepAR univariate time series model on `completed_routes_max` -> 10th/50th/90th percentile auxiliary features
- DSP-type one-hot encoding (DSP 2.0, DSP 1.0, Walker, Transfer, Pinnacle, Pop-Up)
- PowerTransformer + StandardScaler normalization -> 70/30 train/test split

Target label

The model predicts **label = completed_routes_max_0** — the maximum number of routes a DSP can complete in the target prediction week. Outlier clamping is applied for newly-ramping DSPs (min 15 routes enforced after week 7 of tenure).

ModelTrainer — candidate models (5-fold CV, scored by neg-RMSE)

LinearRegression	Baseline linear model, no hyperparameters.
XGBRegressor	GBLinear booster with objective=reg:squarederror. Grid search over n_estimators [150,300], learning_rate [0.01,0.03], reg_lambda [0.0–1.0], reg_alpha [0.001–100].
ElasticNet	Grid search over max_iter [1000,5000], alpha [0.01–100], l1_ratio [0.0–1.0].

Winner selected by lowest **WAPE** (Weighted Absolute Percentage Error) on held-out test set. Best model saved as a pickle alongside PowerTransformer and StandardScaler.

InferenceHandler — predictions

- **predictions_with_incentives** — baseline route capacity forecast
- **predictions_no_incentives** — counterfactual with incentives_0 set to 0
- SHAP linear explanations generated for each inference run
- Post-prediction quality checks and manual override rules applied

File 3 — dsp_target_deployment_optimizer.py

Mixed-Integer Programming deployment optimizer (Gurobi)

Decides which eligible DSPs to deploy to which under-capacity stations in which roles, using the capacity forecasts from the ML model as input.

Eligibility filters (DecisionMaker)

- DSP capacity gap > 0: offered routes < predicted capacity with incentives
- DSP health score >= 50th percentile (dsp_final_metric)
- DSP reliability >= 50th percentile
- DSP currently serves only one station (no splitting)
- Target station is not already under-solved

Decision variables

Binary variable **x[d, r, s]** in {0, 1} indicating whether DSP *d* is deployed in role *r* at station *s*.

Roles: **popup**, **pinnacle**, **transfer**, **newdsps**.

Objective function (extracted from code)

GRB.MINIMIZE

```
m.setObjective(
3.5 * 200 * 7 * sum(capacity_ask)
- 1.5 * 200 * 7 * x.prod(capacity_projection)
+ x.prod(deployment_fixed_cost),
GRB.MINIMIZE
)
```

What this minimizes

Minimize: [network baseline cost] - [savings from capacity filled] + [fixed deployment cost]

Term	Value	Interpretation
3.5 x 200 x 7 x sum(capacity_ask)	Constant (baseline cost)	Cost if all unmet capacity is served by the expensive fallback network (\$3.50/pkg x 200 pkgs/route x 7 days). Worst-case anchor.
- 1.5 x 200 x 7 x x.prod(capacity_projection)	Savings term	Money saved by filling routes with cheaper DSP capacity (\$1.50/pkg). More routes assigned => more savings subtracted from cost.
+ x.prod(deployment_fixed_cost)	Fixed penalty	One-time fixed cost of deploying each DSP to a new station (~\$10k per deployment). Discourages over-assigning marginal targets.

Net effect

The optimizer finds the set of DSP deployments that **maximizes net cost savings** — routing packages through cheaper DSP capacity (\$1.50/pkg) instead of the expensive fallback network (\$3.50/pkg) — while paying the one-time fixed deployment cost per assignment. It will not assign a DSP if the fixed cost exceeds the expected shipping savings.

Constraints

- **One-station constraint:** each eligible DSP can be assigned to at most one station — $x.sum(D, '*', '*') \leq 1$ for all D
- **Station capacity constraint:** total capacity added to any under-served station must not exceed that station's gap — ensures we don't over-assign

Important note on placeholder data

In the current code, **deployment_fixed_cost** and **capacity_projection** are seeded with random normals (mean=\$10,000/sd=\$500 and mean=20/sd=2.5 routes respectively). These are development placeholders, intended to be replaced with actual DCPM model outputs and real contract cost estimates in production.

Model features used in inference (44 features)

DSP 2.0, DSP 1.0, Walker, Internal-Transfer, Pinnacle, Pop-Up	DSP type one-hot encoding
attr_rate_3	Attrition rate (3-week lag)
active_da_count_0	Active delivery associate headcount (current week)
completed_routes_max_max_1, completed_routes_2, completed_routes_max_3	Max completed routes at 1 and 3-week lag; actual routes at 2-week lag
da_experience_1	Average DA experience (1-week lag)
distance	Geographic distance (used for transfer deployments)
driver_affinity_1	Driver-station affinity score (1-week lag)
dsp_final_metric_1, dsp_final_metric_3	Composite DSP performance score (1 and 3-week lag)
dsp_nhscore_1, dsp_nhscore_rank_1	Network health score and rank (1-week lag)
dsp_quality_score_1	DSP quality score (1-week lag)
dsp_tenure_week_0	DSP tenure in weeks (current)
final_route_target_2	Planned route target (2-week lag)
incentives_0	Incentive payments to DSP (current week)
late_cancelled_by_dsp_3	Late cancellations by DSP (3-week lag)
ratio_station_pair_to_station_1	DSP share of station routes (1-week lag)
region_3_5_routes_diff, station_3_5_routes_diff	Regional and station route volume differentials
ratio_station_pair_3_4	Ratio of DSP routes at lag 3 vs lag 4
reliability_1, reliability_3	DSP reliability score (1 and 3-week lag)
requested_routes_max_1	Max routes requested by DSP (1-week lag)
robl_mins_1	Route off-block latency in minutes (1-week lag)
route_stop_agd_sum_1	Assigned-to-delivered route stop aggregate (1-week lag)
spr_3	Stops per route (3-week lag)
station_pair_tenure_0	DSP tenure at this specific station in weeks
sustained_high_das_1	Count of sustained high-performing DAs (1-week lag)
t6_cr_isight_points_1, t6_isight_points_1	iSight compliance points — T6 and critical T6 (1-week lag)
vin_active_rate_1/2/3, vin_branded_rate_1/3, vin_total_1/3	Vehicle fleet utilization and branding rates across lags
vol_share_cycle1_1, vol_share_cycle1_3	Station volume share for cycle 1 deliveries across lags